SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 6

The Prototype Model

A student-facing chapter on prototypes, requirement clarification, user feedback, and evaluating design ideas

Module I: Introduction to Software Engineering
Topic: Software Process Models — Prototype Model
Duration: 1 hour

Running example: College Event Management System (CEMS)

Learn how an early representation of a software system can help users and developers clarify what should be built before investing in the full solution.

1. Learning Objectives

By the end of this chapter, you should be able to:

- Define a prototype and explain the purpose of the Prototype Model.
- Describe a common prototype development cycle.
- Distinguish throwaway prototyping from evolutionary prototyping.
- Explain how user feedback can clarify incomplete or ambiguous requirements.
- Identify benefits, risks, and suitable situations for prototyping.
- Compare the Prototype Model with Waterfall, Incremental, and Spiral approaches.

> **Key idea**
> A prototype makes an idea tangible. Users can interact with or review an early representation, provide feedback, and help the team clarify requirements and design decisions.

2. Why Are Prototypes Useful?

Users often know the problem they want to solve but may find it difficult to describe every screen, interaction, rule, or exception in words. A written requirement such as “the event registration page should be simple” leaves many questions unanswered. What information should appear? How does a student know registration succeeded? What happens when an event is full?

A prototype helps make these questions visible. A developer or designer can create a rough screen, clickable mock-up, or small technical experiment. Users can review it and explain what is correct, confusing, missing, or unnecessary.

Prototypes are especially useful when requirements are unclear, users have different expectations, a new interaction needs evaluation, or the team is uncertain whether a technical approach will work.

3. What Is the Prototype Model?

The **Prototype Model** is a software development approach in which an early version or representation of the proposed system is created to understand requirements, explore design alternatives, or test feasibility. Feedback from evaluating the prototype helps the team decide what the final system should do and how it should behave.

A prototype can be a paper sketch, wireframe, clickable user-interface mock-up, limited working feature, or technical proof of concept. It does not have to be a complete application.

> **Remember**
> A prototype is a learning tool. It helps answer questions before the team commits to the full implementation.

4. Common Prototype Development Cycle

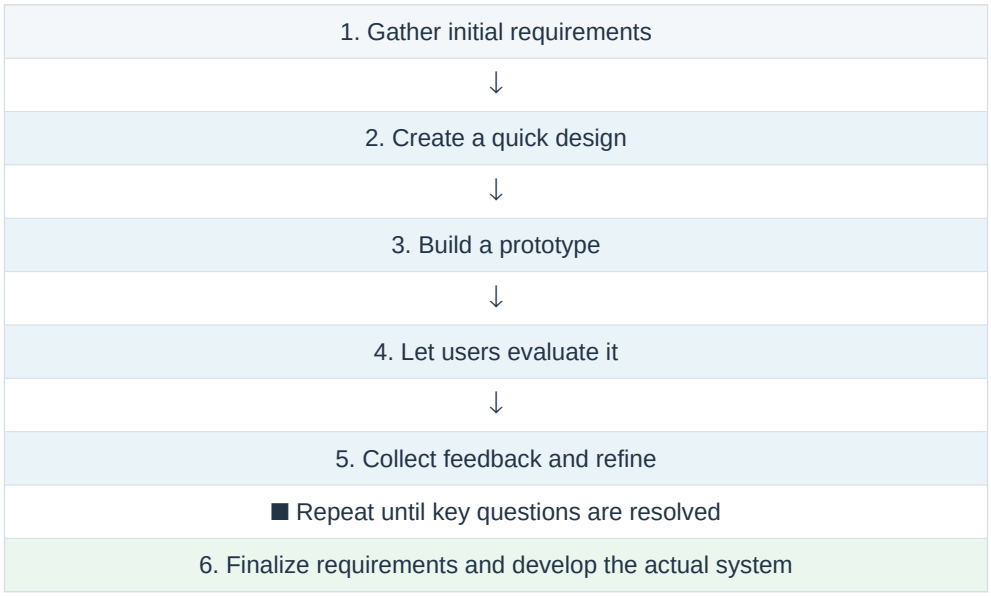

Figure 1. A simplified prototyping cycle. The number of feedback rounds depends on the uncertainty and complexity of the requirements.

Step 1 — Gather Initial Requirements

Identify the main users, their goals, and the most important questions to resolve. Do not assume that every detail is already known. For CEMS, the team may need to learn what students expect to see before registering for an event.

Step 2 — Create a Quick Design

Prepare a simple representation of the proposed solution. This could be a paper sketch, wireframe, or a few linked screens. Focus on the questions being investigated rather than visual polish.

Step 3 — Build the Prototype

Create enough detail for users to evaluate the idea. A user-interface prototype may use sample data and may not connect to a real database. A technical prototype may focus on one difficult capability, such as QR scanning or integration with an external service.

Step 4 — Evaluate with Users

Ask representative users to perform realistic tasks. Observe where they hesitate, what they misunderstand, and what they expect to happen. Specific task-based feedback is usually more useful than asking only whether users “like” the screen.

Step 5 — Refine and Decide

Record feedback, resolve disagreements, and update the prototype. Repeat when important questions remain. When the team has enough evidence, document the agreed requirements and proceed with the selected development approach.

5. Worked Example: Event Registration Prototype

The college wants students to register for workshops. The initial request says: “Students should be able to find an event and register easily.” The team creates a clickable prototype to discover what “easily” means in practice.

Prototype element	Question to evaluate	Possible feedback
Event card	Can students quickly identify relevant events?	Show date, venue, organizer, available seats, and a clear event title.
Event details screen	Is enough information available before registration?	Add eligibility, schedule, capacity, and registration closing time.
Register button	Is the next action obvious?	Use a clear label and show a confirmation step where appropriate.
Confirmation screen	Can the student tell that registration succeeded?	Display confirmation, event details, and cancellation rules.
Full event state	What happens when no seats remain?	Explain that registration is closed and optionally show a waitlist option.

What did the prototype help discover?

- The original requirement was too broad to guide a consistent design.
- Students need event details and availability before committing.
- The system needs clear success, failure, and full-capacity states.
- The team can update the requirements and acceptance criteria based on observed tasks.

Example requirement after feedback
“When a student selects an event with available seats and completes registration, the system shall display a confirmation containing the event name, date, and registration status.”

6. Types of Prototyping

Two common approaches are throwaway prototyping and evolutionary prototyping. The important distinction is what happens to the prototype after it has served its purpose.

A. Throwaway (Rapid) Prototyping

The team builds a prototype to learn about requirements or test an idea, then discards it and develops the production system using the clarified requirements and an appropriate design.

- **Useful when:** the team needs to explore screens, workflows, or alternatives quickly.
- **Example:** a clickable event-registration mock-up is used to validate the workflow; the final application is built separately.
- **Benefit:** early experiments do not force the production system to inherit a rushed design.
- **Caution:** users may assume the prototype is nearly finished, so the team should explain its purpose and limitations.

B. Evolutionary Prototyping

The team begins with a working prototype and improves it repeatedly, gradually evolving it toward the final system. This can be useful when requirements emerge through use and the architecture can support controlled growth.

- **Useful when:** users need to explore working functionality to discover their real needs.

- **Example:** a basic event catalogue is built, evaluated, and expanded with registration and notifications.
- **Benefit:** learning and working functionality can accumulate over time.
- **Caution:** without sound architecture and refactoring, the system may become difficult to maintain.

Compare the two
Throwaway prototyping is primarily for learning and is discarded; evolutionary prototyping is refined into the continuing product. Neither approach removes the need for testing, documentation, and quality controls.

7. Advantages of the Prototype Model

- **Clarifies requirements:** users can react to a concrete representation instead of abstract descriptions alone.
- **Early feedback:** usability problems and missing workflows may be identified sooner.
- **Improved communication:** stakeholders and developers can discuss the same example.
- **Explores alternatives:** multiple designs can be compared before choosing one.
- **Can reduce rework:** important misunderstandings may be resolved before full implementation.

8. Limitations and Risks

- **Unrealistic expectations:** users may mistake a mock-up for a nearly complete product.
- **Scope expansion:** each review may introduce additional requests that need prioritization.
- **Weak architecture risk:** rapidly evolving prototypes can become difficult to maintain if they turn into production without redesign.
- **Incomplete quality:** a prototype may not address security, performance, accessibility, reliability, or error handling.
- **Time and cost:** repeated prototypes can consume resources if objectives and stopping criteria are unclear.

9. When Is Prototyping Appropriate?

Prototyping is especially useful when user-interface needs are unclear, stakeholders disagree about workflows, requirements are incomplete, or a technical question needs a small experiment. It is less useful to build elaborate prototypes for every well-understood feature when the cost exceeds the expected learning.

Good practice
Before building a prototype, state the question it should answer. For example: “Can first-year students complete event registration without help?” Then define how the team will observe and evaluate the result.

10. Comparing Waterfall, Incremental, Spiral, and Prototype Models

Model	Main idea	Typical strength	Main concern
Waterfall	Sequential project phases.	Clear phase outputs when requirements are stable.	Late changes may require rework.
Incremental	Deliver functionality in successive portions.	Earlier usable value and staged feedback.	Integration and regression need attention.
Spiral	Repeated cycles guided by risk analysis.	Explicit attention to high-impact uncertainty.	Risk analysis and cycle management can be costly.
Prototype	Build an early representation to learn and refine requirements/design.	Makes unclear needs visible and testable.	Prototype may be mistaken for production-ready software.

11. Classroom Activity: Prototype a Library Search

A library wants students to search for books, check availability, and reserve a copy. In pairs, sketch three screens: search results, book details, and reservation confirmation.

Then ask a partner to complete the task “Find a book and reserve it.” Record where the partner hesitates, what information is missing, and what you would change in the prototype.

12. Knowledge Check

1. What is a prototype?
2. Why can prototypes help clarify requirements?
3. Describe the common prototype development cycle.
4. How does throwaway prototyping differ from evolutionary prototyping?
5. Give two benefits and two risks of prototyping.
6. Why should a prototype not automatically be released as production software?
7. Name one situation where a prototype is especially useful.
8. What question should a team answer before creating a prototype?

Answer key

1. An early or simplified representation of a proposed system. 2. Users can evaluate a concrete example and reveal missing or misunderstood needs. 3. Gather initial requirements, quick design, build, evaluate, refine, then finalize requirements/develop the system. 4. Throwaway is discarded after learning; evolutionary is improved toward the final product. 5. Benefits: clearer requirements and early feedback; risks: unrealistic expectations and weak architecture/quality. 6. It may lack security, robustness, testing, and operational readiness. 7. An unclear user workflow or uncertain technical capability. 8. What specific question or uncertainty should the prototype help resolve?

13. Key Terms

Term	Meaning
Prototype	An early or simplified representation of a proposed system.
Wireframe	A simplified layout showing the structure and content of a screen.
Mock-up	A visual representation of a user interface, often without full functionality.
Throwaway prototyping	Creating a prototype for learning and then discarding it.
Evolutionary prototyping	Repeatedly refining a working prototype toward the final product.
Proof of concept	A small experiment to test whether an approach is feasible.
Acceptance criteria	Conditions that must be met for a requirement or feature to be accepted.
Scope creep	Uncontrolled growth in project scope or requirements.

14. Homework / Preparation for Day 7

Choose a food-ordering or library application. Write a short prototype plan that includes:

- The user problem or uncertainty you want to investigate.
- The users who should review the prototype.
- The screens or behaviors you will represent.
- Three tasks users should attempt.
- How you will record feedback and decide what to change.

Next chapter
Day 7 will cover the Agile Model, including iterative delivery, collaboration, responding to change, and how Agile differs from the other process models.

Syllabus alignment: Module I — Introduction to Software Engineering; Software Process Models: Prototype. Examples are illustrative teaching scenarios.